

Python Virtual Environment

`.venv` & `.env`



🤔 ¿Por qué se usa? El problema del 'infierno de las dependencias'

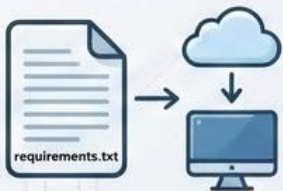


🤔 ¿Para qué sirve? (La solución)

El entorno virtual crea una "burbuja" para cada proyecto. Al usarlo logras:



Aislamiento: Cada proyecto tiene sus propias versiones de librerías sin interferir con los demás.



Portabilidad: Puedes generar un archivo **requirements.txt** con las versiones exactas para que otra persona replique tu entorno fácilmente.



Limpieza: No llenas tu sistema operativo de paquetes que quizás solo usaste una vez.

¿Cómo se usa? (Paso a paso)

Hoy en día, Python trae una herramienta llamada venv por defecto. Aquí están los comandos estándar:

1. Crear el entorno

Abre una terminal en la carpeta de tu proyecto y ejecuta:

Bash

```
# 'env' es el nombre que le daremos a la carpeta, puedes llamarlo como quieras  
python -m venv env
```

2. Activar el entorno

Debes 'entrar' en la burbuja para que Python sepa que debe usar ese entorno.

En Windows:

```
Bash
```

```
.\env\Scripts\activate
```

En macOS / Linux:

```
Bash
```

```
source env/bin/activate
```

Sabrás que está activo porque verás el nombre (**env**) al principio de tu línea de comandos.

3. Instalar librerías

Ahora puedes instalar lo que necesites sin miedo:

```
Bash
```

```
pip install django
```

4. Desactivar

Cuando termines de trabajar, simplemente escribe:

```
Bash
```

```
deactivate
```



Resumen visual de la estructura



Concepto Global (Sin venv)

Ubicación



Carpeta del sistema
(requiere permisos admin)

Versiones



Una sola versión para
toda la PC

Riesgo



Alto (actualizar algo puede
romper otros apps)



Virtual Environment

Ubicación



Carpeta dentro de tu
proyecto

Versiones



Versión 1.0
proyecto



Versión 2.2
proyecto

Una versión específica
por proyecto

Riesgo



Nulo (solo afecta a ese
proyecto)